

Client Scheduling and Participation

Why scheduling matters

- Clients are heterogeneous and intermittent; naïve selection creates bias and slow rounds

Eligibility criteria (examples)

- Sufficient battery and charging state
- Unmetered/high-quality network; device idle and not in active use
- Minimum storage/CPU/GPU/RAM; recent app/version; security/attestation passed

Selection strategies

- Random sampling from eligible pool each round
- Stratified sampling by geography, device class, or data attributes to reduce bias
- Priority/credit systems to ensure fairness and fresh participation
- Cap per-client frequency to avoid overusing fast devices

Availability patterns to consider

- Time zones and diurnal patterns
- Usage behavior (work hours vs. overnight charging)
- Network variability by region/carrier; hardware capability differences

Robustness and performance

- Over-select (invite $K \gg \text{target}$) to tolerate dropouts; accept first N that check in
- Deadlines and partial aggregation; early stopping if enough updates arrive
- Adaptive cohort size: increase when stragglers are common
- Secure eligibility checks; anonymized telemetry for scheduling
- Metrics to monitor
- Round completion time, acceptance rate, dropout rate, coverage/fairness by strata
- Contribution quality/weight (examples seen, loss improvement)

Communication- Efficient Protocols

Core idea: cut bytes per round without hurting accuracy

Techniques

- **Quantization:** send updates in 8/4/2 bits with per-tensor scales
- **Sparsification:** top-k or thresholded updates; error feedback to recover dropped mass
- **Structured updates:** layer-wise or last-layer only; low-rank adapters (LoRA/PEFT)
- **Delta/periodic sync:** send changes since last round; skip small updates

Practicalities

- Clip and add DP noise if required; secure aggregation friendly
- Calibrate k/bit-width; monitor loss drift and add error feedback
- Compress after aggregation when possible to reduce server load



Non-IID Data Challenges

What's hard

- Client data are skewed and imbalanced; distributions drift over time

Diagnose

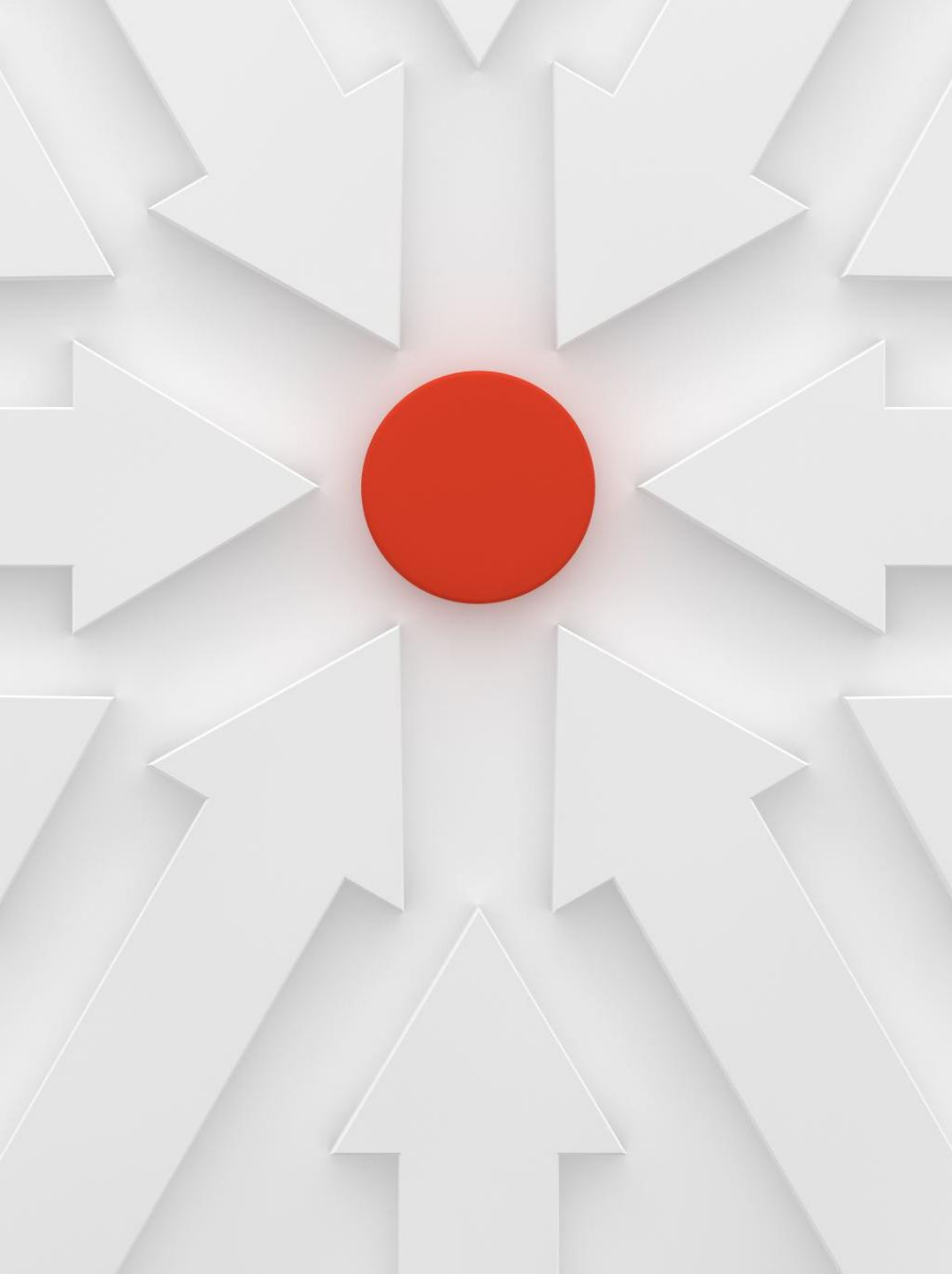
- Compute per-client stats (label/feature histograms, entropy, KS tests)
- Cluster similar clients; track drift across rounds

Adapt aggregation

- Weight by data size, quality, and similarity to global
- Use cohort-wise or personalized aggregation instead of one-size-fits-all

Practical guardrails

- Robustness: clip norms, outlier filtering
- Evaluation: group metrics and worst-case performance



Federated Learning Performance

Convergence factors

- Local steps vs. rounds; data and system heterogeneity; client sampling

Efficiency trade-offs

- More local steps → fewer rounds but risk of drift
- Communication cost vs. accuracy curve

Key metrics

- Rounds to target accuracy; bandwidth per round/total
- Participation rate and straggler impact
- On-device energy/time per round

Performance Optimization

Smart client selection

- Score by data utility, device health, connectivity; diversify to avoid bias

Communication scheduling

- Prefer Wi-Fi + charging; predict availability windows; cache and batch updates

Adaptive training

- Per-device configs: batch size, local steps, precision; auto-tune online
- Early exit for slow/overheating devices

Bandwidth/compute efficiency

- Update compression: sparsify, quantize, delta + server-side momentum
- Partial model updates or adapters; server-side aggregation acceleration

Energy awareness

- Train only on charge or high battery; throttle CPU/GPU; budget per session

Metrics to track

- Time/round, completion rate, bytes/round, on-device energy, accuracy/round

